Week 5 Lab 1: Relational Data Modelling and Persistence

Purpose

In previous weeks, the application stored data in in-memory collections. In this lab, you will replace selected in-memory storage with relational persistence.

The main purpose is not simply to add a database.

You will first examine the information, relationships, and relational representation. You will then implement the resulting data architecture using Entity Framework Core.

Learning Outcomes

By completing the core lab, you should be able to:

- Interpret a simple domain model and relational ERD.
- Implement and migrate a relational model using Entity Framework Core and SQLite.
- Use a DAO to retrieve persistent data through the service layer.
- Display persisted data through an ASP.NET Core MVC view while preserving module boundaries.

Use-Case Descriptions

The Week 5 use-case descriptions will be provided in a separate file.

Read that file before beginning the main implementation. Use the use cases to identify:

- The information the application must store.
- The relationships that must be represented.
- The data required by each operation.
- The business rules that should become database constraints or application rules.
- The queries required by the application.

Do not create tables or persistence classes without relating them to the supplied use cases and domain model.

Files and Project Required

Before starting, make sure you have:

- Your completed Week 4 modular-monolith project.
- The separately supplied Week 5 use-case descriptions.
- Your Week 4 domain-model class diagram.
- Your Week 4 implementation class diagram.
- Your Week 4 sequence diagrams.
- Any tests and design notes retained from previous weeks.

Continue working in the same project or repository retained from Week 4.

You can also use the starter project provided for this week. Make sure to review the provided implementation class diagram before starting.

Part 1: Preserve the Week 4 Baseline

Task 1: Check the Existing Application

Before adding relational persistence:

1. Build and run the Week 4 project.
2. Exercise the existing Student Management use cases.
3. Exercise the Academic Supervisor Management core use cases.
4. Record any incomplete behaviour or known issues.
5. Save or commit a working checkpoint.

```
git status
git add .
git commit -m "Preserve working Week 4 modular monolith"
```

Task 2: Review Current Data Ownership

Identify every in-memory collection in the project.

For each collection, record:

- The owning module.
- The class that owns the collection.
- The domain entities stored in it.
- The services that access it.
- The use cases that depend on it.
- Whether the collection will be replaced by relational persistence in this lab.

Do not remove the in-memory implementation until the relational model and migration have been completed.

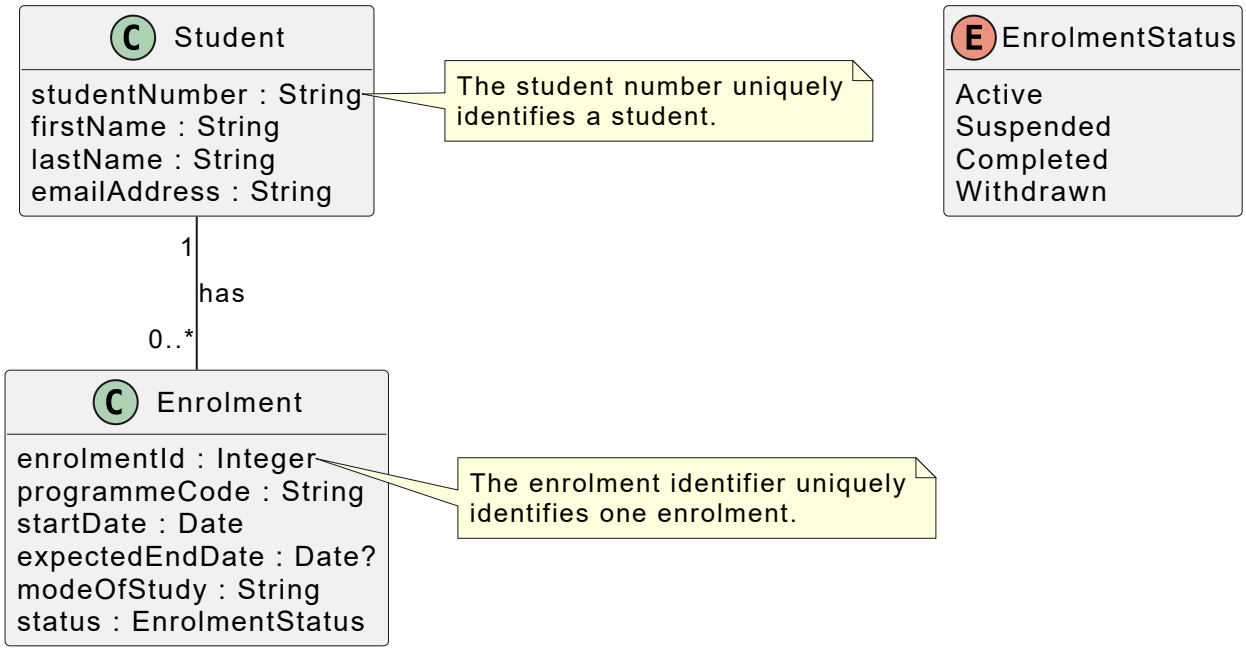
Part 2: Review the Initial Relational Model

Task 3: Understand Student and Enrolment

This week's domain model introduces `Enrolment` as a separate concept from `Student`.

A `Student` represents a person or institutional student record. An `Enrolment` represents a student's participation in a research programme during a defined period. A student may have more than one enrolment over time.

Week 5 Initial Domain Model: Student and Enrolment



Review this model before proceeding further. The relationship means that:

- A student may have no enrolment records.
- A student may have one enrolment.
- A student may have several enrolments over time.
- Every enrolment belongs to exactly one student.

Use the initial domain model to consider:

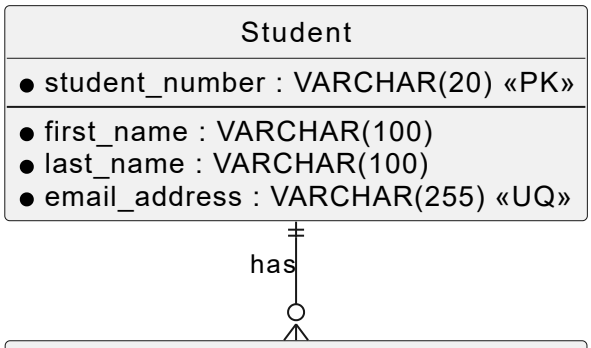
- Why `Student` and `Enrolment` are separate concepts.
- Which values are required or optional.
- Which values should be unique.
- Which Student information remains stable across enrolments.
- Which Enrolment information may differ over time.
- Whether `modeOfStudy` and `status` should use strings or enumerations.
- How previous enrolments are retained and ordered.

Record any changes you would make to the model and explain why. The corresponding initial ERD is embedded in Task 4. Use this task to understand the domain concepts before reviewing their relational representation.

Task 4: Review the Supplied Initial ERD

The following ERD represents `Student` and `Enrolment` in the initial relational model.

Week 5 Initial ERD: Student and Enrolment



Symbol	Meaning
«PK»	Primary key
«FK»	Foreign key
«UQ»	Unique constraint
*	Required column

Review the ERD before proceeding further. Use the ERD to answer:

1. How is a student uniquely identified?
2. How is an enrolment uniquely identified?
3. How does the relational model associate an enrolment with a student?
4. Can a student exist without an enrolment?
5. Can an enrolment exist without a student?
6. Which columns are required?
7. Why is `expected_end_date` optional?
8. Which constraints should be enforced by the database?
9. Which rules should remain in application services?
10. Does the model allow one student to have several enrolments over time?

Compare the supplied ERD with the initial domain model.

If you identify a problem or ambiguity, annotate a copy of the ERD and explain your proposed change. Review the issue with your tutor before configuring the Entity Framework model.

Part 3: Implement the Initial Model with Entity Framework Core

Implement the initial `Student` and `Enrolment` model from Part 2.

Task 5: Add Entity Framework Core

Add the Entity Framework Core packages required for SQLite:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Use versions compatible with the project's target framework. If required, install or update the command-line tool:

```
dotnet tool install dotnet-ef
```

```
dotnet tool update dotnet-ef
```

Check it is available:

```
dotnet ef --version
```

Record the package and tool versions used.

Task 6: Add the Connection String

Add a SQLite connection string to `appsettings.json`:

```
{
  "ConnectionStrings": {
    "StudentManagementDatabase": "Data Source=student-management.db"
  }
}
```

Do not commit credentials or secrets to source control.

Class and File Locations

Use the following locations for the initial Students-module persistence implementation:

```
PgrStudentManagement.Web/
└── Modules/
    └── Students/
        ├── Models/
        │   ├── Student.cs
```

```
│   ├── Enrolment.cs
│   └── EnrolmentStatus.cs
└── Data/
    └── StudentManagementDbContext.cs
```

If your project uses `Domain` and `Persistence` rather than `Models` and `Data`, you may use those names instead. Apply one convention consistently and keep all Students-module classes inside that module.

Task 7: Implement the Initial Entity Classes

Create or update these files:

```
Modules/Students/Models/Student.cs
Modules/Students/Models/Enrolment.cs
Modules/Students/Models/EnrolmentStatus.cs
```

Implement `Student`, `Enrolment`, and `EnrolmentStatus` from Part 2.

The code below shows the properties relevant to the new relational mapping. It is not a complete replacement for the existing class.

```
public class Student
{
    public string StudentNumber { get; set; } = string.Empty;
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string EmailAddress { get; set; } = string.Empty;

    public ICollection<Enrolment> Enrolments { get; set; }
        = new List<Enrolment>();
}
```

```
public class Enrolment
{
    public int EnrolmentId { get; set; }
    public string StudentNumber { get; set; } = string.Empty;
    public string ProgrammeCode { get; set; } = string.Empty;
    public DateTime StartDate { get; set; }
    public DateTime? ExpectedEndDate { get; set; }
    public string ModeOfStudy { get; set; } = string.Empty;
    public EnrolmentStatus Status { get; set; }

    public Student Student { get; set; } = null!;
}
```

```
public enum EnrolmentStatus
{
    Active,
    Suspended,
    Completed,
    Withdrawn
}
```

Adapt namespaces and names to your project. Keep the code, domain model, and ERD consistent.

Task 8: Create and Register the Initial DbContext

Create:

```
Modules/Students/Data/StudentManagementDbContext.cs
```

Add `StudentManagementDbContext` to the Students module:

```
public class StudentManagementDbContext : DbContext
{
    public StudentManagementDbContext(
        DbContextOptions<StudentManagementDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students => Set<Student>();
    public DbSet<Enrolment> Enrolments => Set<Enrolment>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
    }
}
```

Register it in `Program.cs`:

```
builder.Services.AddDbContext<StudentManagementDbContext>(
    options => options.UseSqlite(
        builder.Configuration.GetConnectionString(
            "StudentManagementDatabase")));
```

Task 9: Configure the Initial Relational Mapping

In Task 7, you created:

```
Modules/Students/Models/Student.cs
Modules/Students/Models/Enrolment.cs
Modules/Students/Models/EnrolmentStatus.cs
```

You will now configure how those existing classes map to the initial relational model.

Entity Framework Core supports three configuration approaches:

- Conventions
- Data annotations
- Fluent API

For this initial model, use conventions and data annotations for most of the mapping. Use Fluent API only for decisions that require additional control.

Add Mapping Annotations to Student

Open:

```
Modules/Students/Models/Student.cs
```

Add these namespaces:

```
using System.ComponentModel.DataAnnotations;
using Microsoft.EntityFrameworkCore;
```

Add mapping annotations to the existing `Student` class and properties:

```
[Index(nameof(EmailAddress), IsUnique = true)]
public class Student
{
    [Key]
    [MaxLength(20)]
    public string StudentNumber { get; set; }
        = string.Empty;

    [Required]
    [MaxLength(100)]
    public string FirstName { get; set; }
        = string.Empty;

    [Required]
    [MaxLength(100)]
    public string LastName { get; set; }
        = string.Empty;

    [Required]
```

```
    [MaxLength(255)]
    public string EmailAddress { get; set; }
        = string.Empty;

    public ICollection<Enrolment> Enrolments { get; set; }
        = new List<Enrolment>();
}
```

The class and properties already exist from Task 7. This example shows the annotations to add.

The annotations represent:

- `StudentNumber` as the primary key.
- Maximum lengths matching the ERD.
- Required first name, last name, and email-address values.
- A unique index on `EmailAddress`.

Add Mapping Annotations to Enrolment

Open:

```
Modules/Students/Models/Enrolment.cs
```

Add:

```
using System.ComponentModel.DataAnnotations;
```

Add mapping annotations to the existing `Enrolment` class and properties:

```
public class Enrolment
{
    [Key]
    public int EnrolmentId { get; set; }

    [Required]
    [MaxLength(20)]
    public string StudentNumber { get; set; }
        = string.Empty;

    [Required]
    [MaxLength(20)]
    public string ProgrammeCode { get; set; }
        = string.Empty;

    public DateTime StartDate { get; set; }

    public DateTime? ExpectedEndDate { get; set; }
```

```
    [Required]
    [MaxLength(20)]
    public string ModeOfStudy { get; set; }
        = string.Empty;

    public EnrolmentStatus Status { get; set; }

    public Student Student { get; set; } = null!;
}
```

The class and properties already exist from Task 7. This example shows the annotations to add.

Entity Framework Core conventions can recognise:

- `EnrolmentId` as the primary key.
- `StudentNumber` as the foreign-key property associated with `Student`.
- `Student.Enrolments` and `Enrolment.Student` as a one-to-many relationship.
- `ExpectedEndDate` as optional because its type is `DateTime?`.
- Non-nullable value properties as required.

The `[Key]` annotation on `EnrolmentId` makes the intention explicit, although its name already follows the Entity Framework Core primary-key convention.

Configure the Remaining Mapping Decisions

Open:

```
Modules/Students/Data/StudentManagementDbContext.cs
```

Update `OnModelCreating`:

```
protected override void OnModelCreating(
    ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Enrolment>()
        .HasOne(enrolment => enrolment.Student)
        .WithMany(student => student.Enrolments)
        .HasForeignKey(enrolment =>
            enrolment.StudentNumber)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<Enrolment>()
        .Property(enrolment => enrolment.Status)
        .HasConversion<string>()
        .HasMaxLength(30);
}
```

This Fluent API configuration makes two decisions explicit.

Delete Behaviour

```
.OnDelete(DeleteBehavior.Restrict);
```

This prevents a `Student` record from being deleted while related `Enrolment` records remain. It protects enrolment history from automatic cascade deletion.

Enumeration Storage

```
.HasConversion<string>();
```

This stores `EnrolmentStatus` as readable text:

```
Active
Suspended
Completed
Withdrawn
```

Without this conversion, Entity Framework Core would normally store the enumeration using its numeric value.

Check the Complete Mapping

Confirm that conventions, data annotations, C# nullability, and Fluent API collectively represent:

- `Student.StudentNumber` as the Student primary key.
- `Enrolment.EnrolmentId` as the Enrolment primary key.
- `Enrolment.StudentNumber` as the foreign key.
- A unique index on `Student.EmailAddress`.
- Required student names and email address.
- Required programme code.
- Required start date.
- Optional expected end date.
- Required mode of study.
- Required enrolment status.
- One Student associated with zero or more Enrolments.
- Every Enrolment associated with exactly one Student.
- Restricted deletion of a Student with related Enrolments.
- `EnrolmentStatus` stored as readable text.

You will need to search the EF documentation for some of those points.

Compare the Mapping with the ERD

For each element in the ERD, identify how it is represented using one of these mechanisms:

```
Convention
Data annotation
Fluent API
C# type and nullability
```

For example:

- The Student primary key is represented by `[Key]`.
- The optional expected end date is represented by `DateTime?`.
- The unique email constraint is represented by `[Index]`.
- The relationship is recognised from the navigation and foreign-key properties and made explicit through Fluent API.
- Restricted delete behaviour is configured through Fluent API.
- Enrolment-status conversion is configured through Fluent API.

Document any deliberate difference between the Entity Framework Core model and the initial ERD.

Do not continue to the migration until you can explain how the classes created in Task 7 are mapped to the intended relational model.

Task 10: Create and Inspect the Initial Migration

Create the migration:

```
dotnet ef migrations add InitialStudentEnrolmentModel
```

By default, EF Core creates a `Migrations` directory in the project containing `StudentManagementDbContext`. Keep these generated migration classes with the Students-module persistence project or area. If your solution uses one web project, the expected location is typically:

```
PgrStudentManagement.Web/Migrations/
```

If the context and startup application are in different projects, use the appropriate `--project` and `--startup-project` options.

Before applying the migration, confirm that it creates:

- `Student` and `Enrolment` tables.
- Both primary keys.
- The Enrolment-to-Student foreign key.
- The unique email index.
- The expected required and optional columns.

If it differs from the ERD, correct the mapping before continuing.

Task 11: Create, Seed, and Test the Initial Database

Apply the initial migration:

```
dotnet ef database update
```

This creates the SQLite database using the schema represented by the initial migration.

Inspect the Created Database

Open the SQLite database using a suitable database inspection tool.

Confirm that it contains:

- The `Student` table.
- The `Enrolment` table.
- The primary key on `Student.StudentNumber`.
- The primary key on `Enrolment.EnrolmentId`.
- The foreign key from `Enrolment.StudentNumber` to `Student.StudentNumber`.
- The unique index on `Student.EmailAddress`.
- A nullable `ExpectedEndDate` column.
- The Entity Framework migrations-history table.

Do not continue until the generated schema is consistent with the initial ERD.

Create the Seed Class

Create:

```
Modules/Students/Data/StudentDataSeeder.cs
```

Add the following code:

```
using Microsoft.EntityFrameworkCore;
using PgrStudentManagement.Web.Modules.Students.Models;

namespace PgrStudentManagement.Web.Modules.Students.Data;

public static class StudentDataSeeder
{
    public static async Task SeedAsync(
        StudentManagementDbContext context)
    {
        if (await context.Students.AnyAsync())
        {
            return;
        }
```

```
var studentOne = new Student
{
    StudentNumber = "S100001",
    FirstName = "Alex",
    LastName = "Morgan",
    EmailAddress = "alex.morgan@example.test"
};

var studentTwo = new Student
{
    StudentNumber = "S100002",
    FirstName = "Sam",
    LastName = "Patel",
    EmailAddress = "sam.patel@example.test"
};

var studentThree = new Student
{
    StudentNumber = "S100003",
    FirstName = "Jordan",
    LastName = "Taylor",
    EmailAddress = "jordan.taylor@example.test"
};

var studentFour = new Student
{
    StudentNumber = "S100004",
    FirstName = "Casey",
    LastName = "Wilson",
    EmailAddress = "casey.wilson@example.test"
};

var enrolmentOne = new Enrolment
{
    EnrolmentId = 1,
    StudentNumber = studentOne.StudentNumber,
    ProgrammeCode = "PHD-COMP",
    StartDate = new DateTime(2024, 10, 1),
    ExpectedEndDate = new DateTime(2027, 9, 30),
    ModeOfStudy = "Full-time",
    Status = EnrolmentStatus.Active
};

var enrolmentTwo = new Enrolment
{
    EnrolmentId = 2,
    StudentNumber = studentTwo.StudentNumber,
    ProgrammeCode = "MPHIL-COMP",
    StartDate = new DateTime(2022, 10, 1),
    ExpectedEndDate = new DateTime(2024, 9, 30),
    ModeOfStudy = "Full-time",
    Status = EnrolmentStatus.Completed
};

var enrolmentThree = new Enrolment
```

```
        {
            EnrolmentId = 3,
            StudentNumber = studentTwo.StudentNumber,
            ProgrammeCode = "PHD-COMP",
            StartDate = new DateTime(2024, 10, 1),
            ExpectedEndDate = new DateTime(2030, 9, 30),
            ModeOfStudy = "Part-time",
            Status = EnrolmentStatus.Active
        };

        var enrolmentFour = new Enrolment
        {
            EnrolmentId = 4,
            StudentNumber = studentThree.StudentNumber,
            ProgrammeCode = "PHD-DESIGN",
            StartDate = new DateTime(2025, 10, 1),
            ExpectedEndDate = null,
            ModeOfStudy = "Full-time",
            Status = EnrolmentStatus.Suspended
        };

        context.Students.AddRange(
            studentOne,
            studentTwo,
            studentThree,
            studentFour);

        context.Enrolments.AddRange(
            enrolmentOne,
            enrolmentTwo,
            enrolmentThree,
            enrolmentFour);

        await context.SaveChangesAsync();
    }
}
```

The `.test` email-address domain is used so that the seed data does not contain the addresses of real people.

The seed data demonstrates:

- A student with one enrolment.
- A student with two enrolments over time.
- A completed enrolment followed by an active enrolment.
- Full-time and part-time modes of study.
- An active enrolment.
- A completed enrolment.
- A suspended enrolment.
- An enrolment with no expected end date.

Run the Seeder During Application Startup

Open:

```
Program.cs
```

After creating the application:

```
var app = builder.Build();
```

and before:

```
app.Run();
```

create a dependency-injection scope and run the seeder:

```
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider
        .GetRequiredService<StudentManagementDbContext>();

    await context.Database.MigrateAsync();
    await StudentDataSeeder.SeedAsync(context);
}
```

Ensure that `Program.cs` imports the relevant namespaces:

```
using Microsoft.EntityFrameworkCore;
using PgrStudentManagement.Web.Modules.Students.Data;
```

`StudentManagementDbContext` is registered as a scoped dependency. Creating a scope allows the application startup code to resolve and use it correctly.

`MigrateAsync()` applies any outstanding migrations before the seed operation runs.

Do not use `EnsureCreated()` in this project because the database schema is being managed through Entity Framework Core migrations.

Understand the Duplicate-Data Check

The seeder begins with:

```
if (await context.Students.AnyAsync())
{
    return;
}
```

This prevents the same seed records from being inserted every time the application starts.

For this lab, the check is sufficient because the seed data is inserted as one initial dataset.

If your database already contains partial or unrelated student data, delete the development database and recreate it before testing the seeder:

```
dotnet ef database drop --force
dotnet ef database update
```

The database-drop command deletes all data in the development database. Use it only for the local teaching database.

Run the Application

Build and run the application:

```
dotnet build
dotnet run
```

When the application starts, it should:

1. Apply outstanding migrations.
2. Check whether Student records already exist.
3. Insert the seed data if the database is empty.
4. Continue starting the web application.

Inspect the Seeded Data

Open the database and confirm that the `Student` table contains:

```
S100001
S100002
S100003
S100004
```

Confirm that the `Enrolment` table contains four records.

Check that:

- `S100001` has one enrolment.
- `S100002` has two enrolments.
- `S100003` has one enrolment.
- `S100004` displays **No enrolments found**.
- Every enrolment references an existing student.
- Enrolment 4 has no expected end date.
- The status values are stored as readable text.

Task 12: Display the Seeded Enrolment Data Through the Application

The seed data should now be viewed through the application rather than through a temporary database query.

Implement this dependency path:

```
StudentsController
    -> StudentService
        -> EnrolmentDao
            -> StudentManagementDbContext
                -> SQLite database
```

Create EnrolmentDao

Create:

```
Modules/Students/Data/Daos/EnrolmentDao.cs
```

Use:

```
using Microsoft.EntityFrameworkCore;
using PgrStudentManagement.Web.Modules.Students.Models;

namespace PgrStudentManagement.Web.Modules.Students.Data.Daos;

public class EnrolmentDao
{
    private readonly StudentManagementDbContext _context;

    public EnrolmentDao(StudentManagementDbContext context)
    {
        _context = context;
    }

    public async Task<IReadOnlyList<Enrolment>>
        GetEnrolmentsForStudentAsync(string studentNumber)
    {
        return await _context.Enrolments
            .AsNoTracking()
```

```
            .Where(enrolment =>
                enrolment.StudentNumber == studentNumber)
            .OrderBy(enrolment => enrolment.StartDate)
            .ToListAsync();
    }
}
```

Register it as a scoped dependency in `Program.cs`:

```
builder.Services.AddScoped<EnrolmentDao>();
```

The DAO encapsulates the EF Core query. It does not select views, return MVC results, or make presentation decisions.

Update StudentService

Open:

```
Modules/Students/Services/StudentService.cs
```

Add `EnrolmentDao` to the existing constructor dependencies. Retain the service's existing operations.

For example:

```
private readonly EnrolmentDao _enrolmentDao;

public StudentService(
    StudentStorage studentStorage,
    EnrolmentDao enrolmentDao)
{
    _studentStorage = studentStorage;
    _enrolmentDao = enrolmentDao;
}
```

Adapt the constructor to the dependencies in your implementation.

Add:

```
public Task<IReadOnlyList<Enrolment>>
    GetEnrolmentsForStudentAsync(string studentNumber)
{
    return _enrolmentDao
        .GetEnrolmentsForStudentAsync(studentNumber);
}
```

The service coordinates the use case and delegates relational data access to the DAO.

Update StudentsController

Open:

```
Modules/Students/Controllers/StudentsController.cs
```

Add:

```
public async Task<IActionResult> Enrolments(
    string studentNumber)
{
    var enrolments = await _studentService
        .GetEnrolmentsForStudentAsync(studentNumber);

    ViewData["StudentNumber"] = studentNumber;

    if (enrolments.Count == 0)
    {
        ViewData["Message"] = "No enrolments found";
    }

    return View(enrolments);
}
```

For the core implementation, an empty enrolment query produces **No enrolments found**. Distinguishing an unknown student from a known student with no enrolments is an optional refinement.

The controller selects the presentation response. It must not query `StudentManagementDbContext` or call `EnrolmentDao` directly.

Create the Razor View

Create:

```
Views/Students/Enrolments.cshtml
```

Use:

```
@model IReadOnlyList<PgrStudentManagement.Web.Modules.Students.Models.Enrolment>

@{
    ViewData["Title"] = "Student Enrolments";
}

<h1>Student Enrolments</h1>
```

```
<p>Student number: @ViewData["StudentNumber"]</p>

@if (Model.Count == 0)
{
    <p class="alert alert-info">
        @(ViewData["Message"] ?? "No enrolments found")
    </p>
}
else
{
    <table class="table">
        <thead>
            <tr>
                <th>Programme</th>
                <th>Start date</th>
                <th>Expected end date</th>
                <th>Mode of study</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody>
        @foreach (var enrolment in Model)
        {
            <tr>
                <td>@enrolment.ProgrammeCode</td>
                <td>@enrolment.StartDate.ToString("dd MMMM yyyy")</td>
                <td>
                    @(enrolment.ExpectedEndDate?.ToString(
                        "dd MMMM yyyy") ?? "Not recorded")
                </td>
                <td>@enrolment.ModeOfStudy</td>
                <td>@enrolment.Status</td>
            </tr>
        }
        </tbody>
    </table>
}
```

Add Navigation

Add a link from the appropriate Student list or details view:

```
<a asp-action="Enrolments"
   asp-route-studentNumber="@student.StudentNumber">
    View enrolments
</a>
```

Adapt `student` to the model or loop variable used by your view.

Verify the Seeded Data in the Browser

Run the application and use the HTML page to confirm that:

- `S100001` displays one enrolment.
- `S100002` displays two enrolments in chronological order.
- `S100003` displays an enrolment with no expected end date.
- Programme code, dates, mode of study, and status are displayed.
- A student without enrolments produces the supplied empty-result response.
- The controller calls the service.
- The service calls `EnrolmentDao`.
- `EnrolmentDao` calls `StudentManagementDbContext`.

Task 13: Verify Persistence and Existing Behaviour

Stop and restart the application, then repeat the enrolment-viewing use case in the browser.

Confirm that:

- The seeded records remain available after restart.
- The seeder does not insert duplicate records.
- Enrolments remain in chronological order.
- Existing Student Management behaviour still works.
- Academic Supervisor Management continues to use its existing in-memory storage.
- Module ownership and dependency direction remain clear.

Core Completion Checklist

- [ ] I interpreted the initial domain model and ERD.
- [ ] I configured EF Core and SQLite.
- [ ] I created and mapped the entity classes.
- [ ] I created, inspected, and applied the migration.
- [ ] I seeded and inspected the database.
- [ ] I created and registered `EnrolmentDao`.
- [ ] I updated `StudentService` and `StudentsController`.
- [ ] I created the enrolments Razor view.
- [ ] I viewed the seeded enrolment data in the browser.
- [ ] I verified the empty-result alternative.
- [ ] Data remains available after restart.
- [ ] Existing Student and Academic Supervisor behaviour still works.

Extension Tasks

The following activities are optional and are not expected to be completed within the two-hour core lab. Complete them in the order agreed with your tutor.

Extension 1: Update the Existing Implementation Class Diagram

Update the Week 4 implementation class diagram to show the relational-persistence changes completed during the core lab.

Add, where present:

- `StudentManagementDbContext`
- `DbSet<Student>`
- `DbSet<Enrolment>`
- `EnrolmentDao`
- The updated `StudentService`
- The updated `StudentsController`
- `StudentDataSeeder`
- The SQLite database

Also show that:

- `StudentManagementDbContext` belongs to the Students module.
- `EnrolmentDao` depends on `StudentManagementDbContext`.
- `StudentService` depends on `EnrolmentDao`.
- `StudentsController` does not depend directly on the DAO or context.
- The Academic Supervisors module continues to use its existing in-memory storage at this point.

Use the actual class names, namespaces, fields, methods, and constructor dependencies from your implementation.

Keep the domain class diagram, ERD, and implementation class diagram as separate artefacts.

Extension 2: Refactor the Remaining Student Use Cases

Replace the remaining Students-module in-memory data access with relational persistence.

For each existing Student use case:

1. Identify the required query or update.
2. Add or reuse a Students-module DAO operation.
3. Update the service to use the DAO.
4. Keep the controller independent of `DbContext` and DAOs.
5. Test the main and supplied alternative flows.
6. Confirm that data remains available after restart.

Remove the old Students-module in-memory storage only after all affected use cases work against the database. The Academic Supervisors module may remain in memory unless you select it in Extension 3.

Extension 3: Select and Implement a New Relational Module

The complete model introduces two candidate business modules:

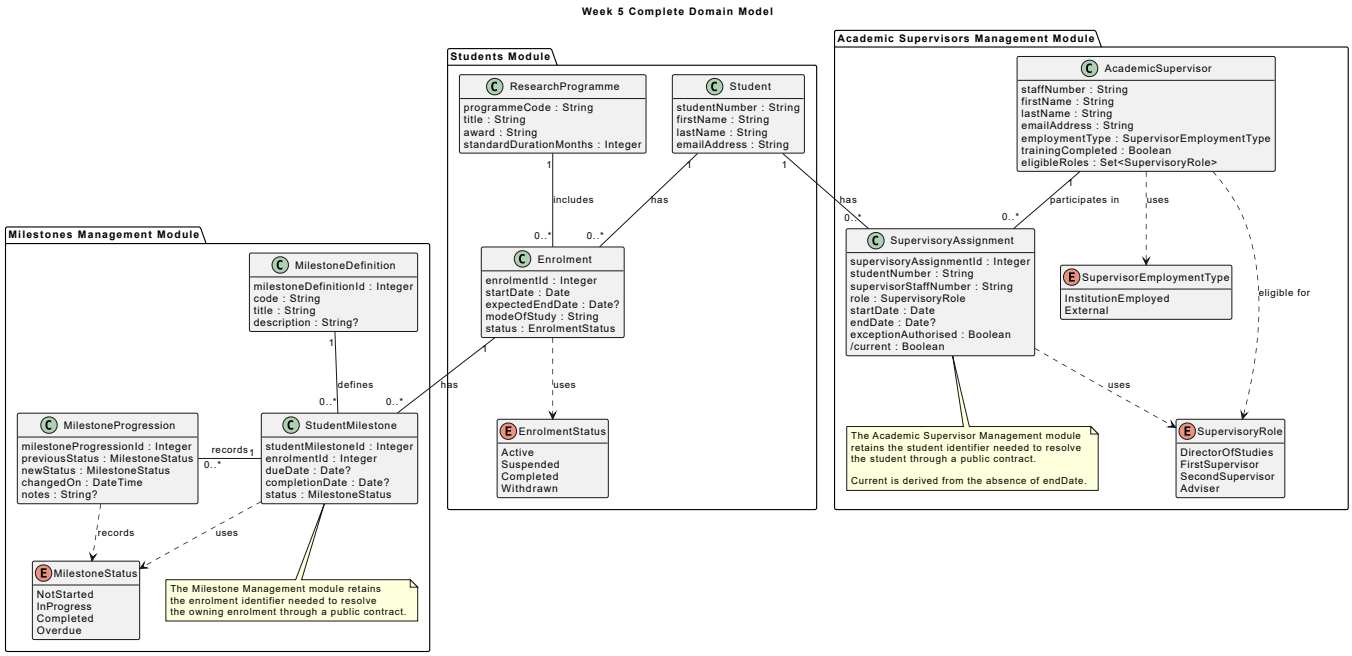
- **Milestone Management**
- **Academic Supervisor Management**

Select **one** of these modules to implement with relational persistence.

You are not required to implement both modules. Your selected module should have its own domain classes, DAO, service operations, persistence mapping, and module-owned relational tables.

Before selecting a module, review the complete domain model, ERD, and draft implementation diagram below.

Complete Domain Model



The complete model contains three business modules:

```
Students Management
Milestones Management
Academic Supervisors Management
```

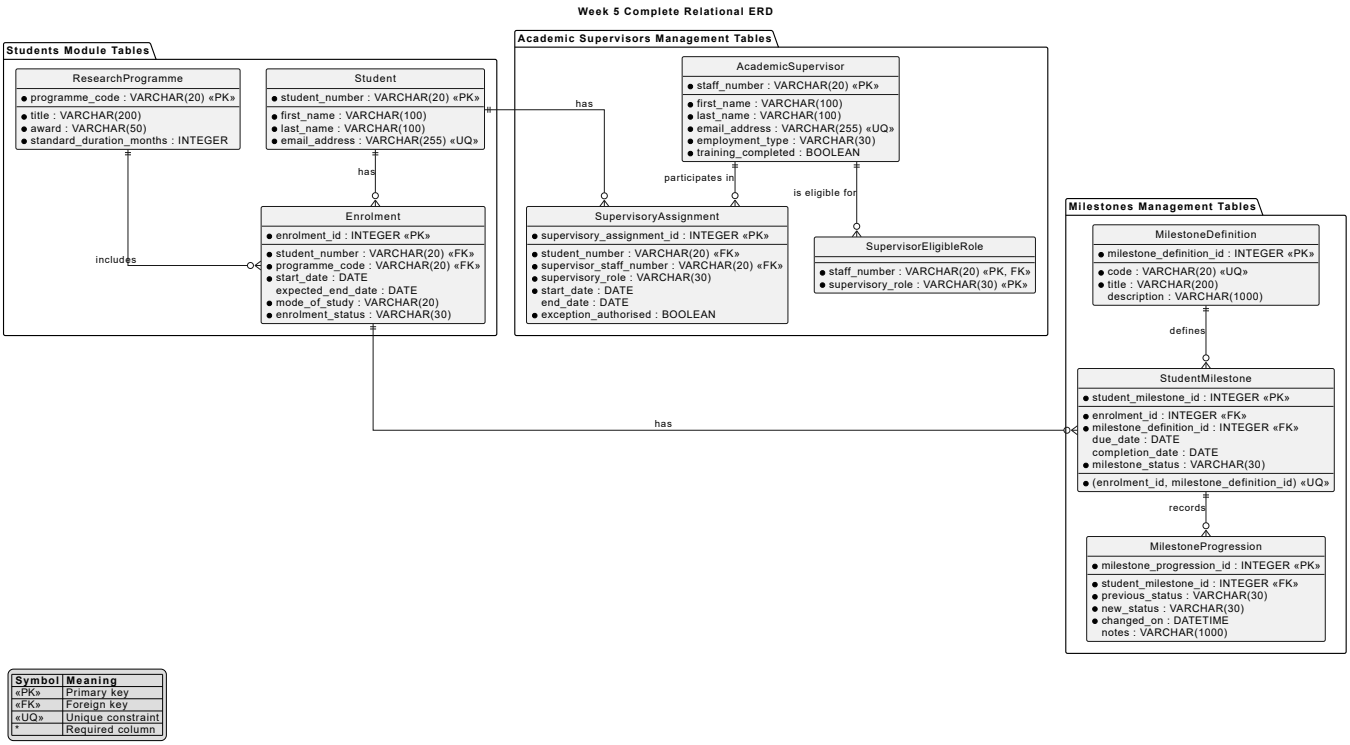
The Students module owns `Student`, `ResearchProgramme`, and `Enrolment`.

The Milestones Management module owns `MilestoneDefinition`, `StudentMilestone`, `MilestoneProgression`, and `MilestoneStatus`. A student milestone refers to an enrolment owned by the Students module.

The Academic Supervisors Management module owns `AcademicSupervisor`, `SupervisoryAssignment`, `SupervisorEmploymentType`, and `SupervisoryRole`. A supervisory assignment refers to a student owned by the Students module.

The cross-module references use stable identifiers rather than transferring ownership of the complete Student or Enrolment entity.

Complete Relational ERD



Review the Relational Model

Identify:

- The tables owned by each module.
- Primary and foreign keys.
- Required and optional columns.
- Unique constraints.
- How `eligibleRoles` is represented relationally.
- Why `/current` does not require a separate stored column.
- How the Milestone Management module refers to an enrolment.
- How Academic Supervisor Management refers to a student.

The ERD shows the complete physical relationship model in one database. Module ownership still applies even when tables share the same SQLite database.

Choose a Module

Choose one implementation path.

Path A: Milestone Management

Implement:

- `MilestoneDefinition`
- `StudentMilestone`
- `MilestoneStatus`
- `MilestoneProgression`
- Module-owned EF mappings
- A milestone DAO
- The service operations required by `W5-UC02`: View Enrolment Milestones

Create these model classes under:

```
Modules/Milestones/Models/
```

Create persistence classes under:

```
Modules/Milestones/Data/
Modules/Milestones/Data/Daos/
```

Path B: Academic Supervisor Management

Move the existing Academic Supervisor in-memory persistence to the relational model.

Implement or update:

- `AcademicSupervisor`
- `SupervisorEmploymentType`
- `SupervisoryRole`
- `SupervisoryAssignment`
- The relational representation of eligible roles
- Module-owned EF mappings
- Academic Supervisor and assignment DAOs
- Existing supervisor services so they use DAOs

Keep model classes under:

```
Modules/AcademicSupervisors/Models/
```

Create persistence classes under:

```
Modules/AcademicSupervisors/Data/
Modules/AcademicSupervisors/Data/Daos/
```

Extend the Database

Add only the tables required by your selected module, together with any required `ResearchProgramme` change to `Enrolment`.

Create a second migration with a name that identifies your path, for example:

```
dotnet ef migrations add AddMilestoneManagementModule
```

or:

```
dotnet ef migrations add AddAcademicSupervisorManagementModule
```

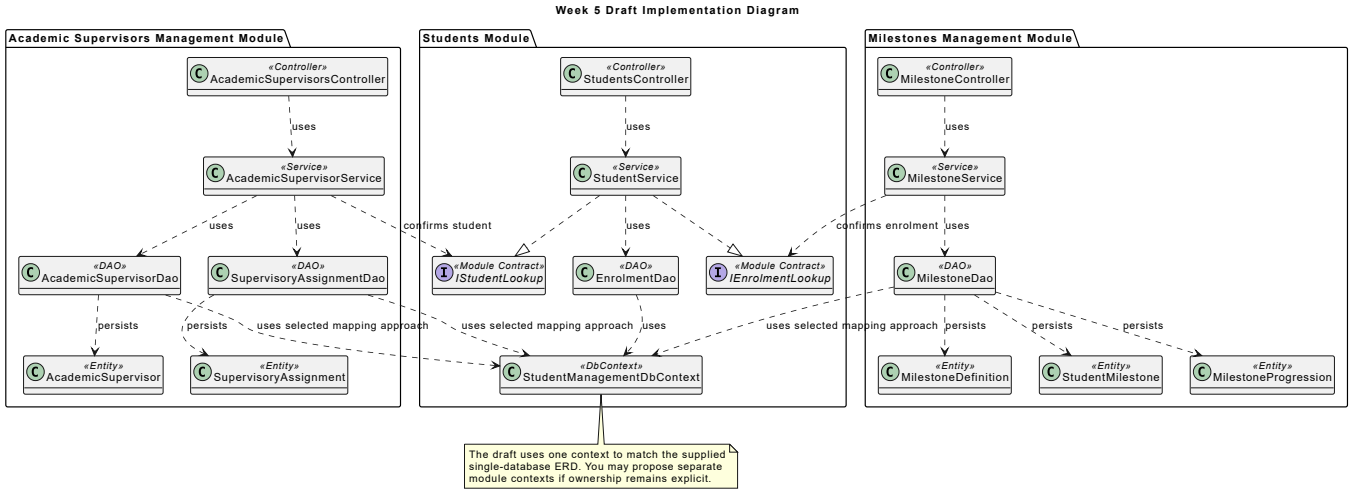
Inspect and apply the migration:

```
dotnet ef database update
```

Verify that the initial Student and Enrolment data remains available.

Draft Implementation Diagram

The following diagram provides a starting point. It shows both candidate modules so that their potential dependencies are visible. Complete only the detail required for your selected module.



Complete the Diagram for Your Chosen Module

Copy the draft implementation diagram and complete the selected module using your actual code.

Add:

- Actual namespaces and class names.
- Entity attributes relevant to persistence.
- DAO operations.
- Service operations.
- Controller actions.
- Constructor dependencies.
- `DbSet<T>` properties or the selected module context.
- Public module contracts.
- Result and DTO types, if used.
- The SQLite database dependency.

For the module you did not select, retain only a labelled placeholder or remove its code-level details. Do not present unimplemented classes as completed code.

Check that:

- Controllers do not use DAOs or `DbContext` directly.
- Services coordinate use-case and business behaviour.
- DAOs encapsulate EF Core queries and updates.
- Cross-module access uses public contracts rather than another module's DAO or context internals.
- The completed diagram matches the implementation.

Extension 4: Reverse-Engineer the Database

Use an appropriate tool to reverse-engineer the database schema into code or a diagram. Compare the result with the complete domain model, ERD, and implementation class diagram.

Record which business meaning can and cannot be recovered from the schema alone.

Extension 5: Introduce a Data Transfer Object

A Data Transfer Object, or DTO, carries selected data across a boundary without exposing the complete entity graph.

Create a DTO inside your selected module's public contracts or DTO directory.

For the Students module, possible files include:

```
Modules/Students/Contracts/Dtos/EnrolmentSummaryDto.cs
Modules/Students/Contracts/Dtos/EnrolmentDetailsDto.cs
```

For a selected new module, use its equivalent contracts directory.

Record:

- The boundary the DTO crosses.
- Why the complete entity is not required.
- Where the mapping occurs.
- Whether projection occurs in the database query or after retrieval.

A DTO transfers selected data. It should not contain database-access or business-rule behaviour.

Reflection Questions

1. Which mapping decisions use conventions, annotations, or Fluent API?
2. What responsibility belongs to the DAO rather than the service or controller?
3. How did relational persistence change the Students module without changing its public behaviour?
4. If you attempted Extension 3, why did you select that module?

Keeping the Project for Lab 2

Keep the following for Week 5 Lab 2:

- The completed relational implementation.
- The revised domain model.
- The supplied and annotated ERDs.
- Entity Framework configuration and migrations.
- DAO classes and any DTOs created as extensions.
- The updated implementation class diagram.
- Mapping and comparison notes.
- Tests and design decisions.

Before finishing, build the complete project and save or commit a working checkpoint:

```
git status
git add .
git commit -m "Complete Week 5 relational data architecture"
```